# Observabilidad y Login Seguro en Aplicaciones Web: Arquitectura con FIDO2/WebAuthn, OAuth 2.0 y OpenTelemetry sobre Spring Boot y React

Yojhan Toro, Angel Cuervo, David Castro, Ivan Cubillos
*Seminario Seguridad y Privacidad de TI*
*Escuela Colombiana de Ingeniería Julio Garavito*
Bogotá, Colombia
{yojhan.toro-r, angel.cuervo, david.csierra, ivan.cubillos-v}@mail.escuelaing.edu.co

***Resumen*—La gestión de identidades digitales y la observabilidad del comportamiento de las aplicaciones representan dos pilares inseparables de la seguridad en software contemporáneo. Los esquemas de autenticación basados en contraseñas estáticas son sistémicamente vulnerables frente a phishing, credential stuffing y toma de control de cuentas (ATO), vectores responsables de pérdidas globales estimadas en 9,5 billones de dólares anuales [1]. De forma paralela, la ausencia de observabilidad estructurada impide detectar intrusiones basadas en credenciales legítimas robadas, brecha catalogada por OWASP como A09:2021 [6]. Este artículo presenta el diseño de una arquitectura que aborda ambas problemáticas de forma integrada: un sistema de login seguro sin contraseñas fundamentado en FIDO2/WebAuthn, OAuth 2.0 con PKCE y JWT gestionado por Spring Security, combinado con una capa de observabilidad transversal basada en OpenTelemetry que exporta telemetría a Grafana y atraviesa todos los componentes del sistema. La arquitectura se implementa sobre Spring Boot (backend) y React/Vite (frontend), desplegada en infraestructura cloud (AWS/Azure). Se evalúa la conformidad frente al NIST SP 800-63-4 [4], el OWASP Top 10 [6] y el marco normativo colombiano (Leyes 1273 de 2009 y 1581 de 2012).**

***Index Terms*—autenticación, passkeys, FIDO2, WebAuthn, OAuth 2.0, PKCE, JWT, Spring Security, OpenTelemetry, Grafana, observabilidad, seguridad de aplicaciones web, NIST SP 800-63-4**

## I. Introducción

La transformación digital ha disuelto los perímetros de red tradicionales. La proliferación del cómputo en la nube, los ecosistemas de dispositivos móviles y las arquitecturas de microservicios han fragmentado la infraestructura hasta el punto en que cada componente representa una superficie de ataque individual [24]. En este entorno, la **identidad del usuario** se ha convertido en la nueva frontera de control, y la **observabilidad del sistema** en el mecanismo que permite detectar cuando esa frontera ha sido comprometida.

Estas dos problemáticas son complementarias e inseparables. El inicio de sesión determina quién interactúa con el sistema, pero si ese proceso ocurre con credenciales válidas pero robadas, ningún indicador clásico de salud del sistema registrará anomalía alguna. La reutilización masiva de credenciales (más del 50 % de los usuarios reutiliza la misma contraseña en al menos cinco cuentas [13]) convierte el *credential stuffing* en un vector industrializado. Al mismo tiempo, OWASP reconoce que las "Fallas en el Registro y Monitoreo de Seguridad" (A09:2021) son el mecanismo que permite a los atacantes operar sin ser detectados durante semanas o meses tras el compromiso inicial [6].

Este artículo propone una arquitectura de **Seguridad por Diseño** [7] que responde a ambas problemáticas de forma integrada: passkeys FIDO2/WebAuthn como mecanismo primario de autenticación sin contraseña, OAuth 2.0 con PKCE y JWT con Spring Security para la gestión del ciclo de vida de sesiones, y OpenTelemetry como capa de observabilidad transversal que cruza todos los componentes del sistema y exporta telemetría a Grafana para análisis y alertas en tiempo real.

## II. Marco Teórico

### II-A. Fundamentos de Seguridad de la Información

La seguridad de la información descansa sobre la tríada CIA (*Confidentiality, Integrity, Availability*), a la que los marcos modernos agregan el **No Repudio** como cuarto pilar [31]:

- **Confidencialidad:** Acceso restringido a entidades autorizadas, garantizado mediante AES-256 en reposo y TLS 1.3 en tránsito.
- **Integridad:** Inmutabilidad respaldada por funciones hash criptográficas (SHA-256 o superior) y firmas digitales.
- **Disponibilidad:** Acceso oportuno para usuarios legítimos, mitigando amenazas DoS/DDoS.
- **No Repudio:** Garantía criptográfica de que una acción no puede ser negada por quien la originó.

La **Arquitectura Zero Trust** [5], [24] materializa la Defensa en Profundidad con el axioma "*nunca confiar, siempre*

*verificar*": toda solicitud debe autenticarse y autorizarse dinámicamente, independientemente de la ubicación topológica del solicitante.

### *II-B. Autenticación Digital: Conceptos Fundamentales*

*II-B1. Contraseñas Estáticas:* El modelo tradicional almacena un hash de la contraseña en el servidor. La vulnerabilidad estructural radica en que ambos extremos deben conocer el mismo secreto. Una brecha en la base de datos compromete todas las cuentas, especialmente con algoritmos obsoletos (MD5, SHA-1 sin *salting*) [17].

*II-B2. Autenticación Multifactor (MFA):* La MFA combina algo que el usuario sabe, posee o es. Los códigos OTP por SMS son susceptibles al secuestro SS7, la clonación de SIM y los ataques AiTM[1] [14]. Los estándares FIDO U2F introdujeron llaves hardware cuya respuesta criptográfica está vinculada al *Origin* verificado del sitio, haciendo imposible la autenticación en dominios falsos [13].

*II-B3. Passkeys y FIDO2/WebAuthn:* Los **passkeys** eliminan la contraseña como factor humano. Fundamentados en la especificación WebAuthn del W3C y el protocolo CTAP2 de la FIDO Alliance [3], operan mediante criptografía asimétrica. Durante el **registro**, el dispositivo genera un par de claves único por dominio: la clave privada queda en el enclave seguro del dispositivo (Secure Enclave en iOS/macOS, TPM en Windows) y la clave pública se almacena en el servidor. Durante la **autenticación**, el servidor emite un *challenge* aleatorio que el dispositivo firma localmente tras verificación biométrica; el servidor valida la firma con la clave pública registrada. La clave privada **jamás abandona el dispositivo**, erradicando algorítmicamente el *credential stuffing*.

*II-B4. JSON Web Tokens (JWT):* Tras validar las credenciales, el servidor emite un JWT firmado con claims sobre identidad y permisos. Los servicios validan matemáticamente la autenticidad del token sin consultas síncronas a la base de datos central, maximizando la escalabilidad en arquitecturas distribuidas [13].

*II-B5. OAuth 2.0 Authorization Code con PKCE:* Los clientes públicos (SPAs, apps móviles) no pueden almacenar un *client_secret* de forma segura. La extensión **PKCE** (*Proof Key for Code Exchange*) resuelve esta limitación mediante entropía dinámica [14]: el cliente genera un `code_verifier`, calcula su hash SHA-256 (`code_challenge`) y lo envía al servidor de autorización. Al canjear el código, revela el `code_verifier` original, que el servidor verifica. Un atacante que intercepte el código de autorización no conoce el `code_verifier` y el canje fracasa. PKCE es el estándar obligatorio en 2025 para todos los clientes públicos [15].

### *II-C. Observabilidad: Conceptos Fundamentales*

*II-C1. Monitoreo Tradicional vs. Observabilidad:* El monitoreo tradicional opera sobre métricas predefinidas (CPU, memoria, latencia) y emite alertas ante umbrales estáticos. Es inherentemente reactivo: detecta "qué falló" solo ante problemas conocidos. Cuando un atacante accede con credenciales válidas pero robadas, estos indicadores no registran anomalía alguna. La **observabilidad** es una propiedad arquitectónica diseñada desde la concepción del sistema: permite inferir su estado interno a partir de la correlación de sus salidas externas, respondiendo "*por qué falló*" y descubriendo anomalías desconocidas [18], [19].

*II-C2. El Marco MELT:* La observabilidad se fundamenta en cuatro tipos de datos [18]: **Métricas** cuantitativas del sistema, **Eventos** discretos con contexto de negocio, **Logs** cronológicos estructurados y **Trazas** distribuidas que correlacionan la ejecución de una solicitud a través de múltiples servicios. En el contexto de autenticación, las trazas son el pilar más crítico: permiten asociar la identidad del usuario con cada acción post-login y detectar comportamientos anómalos que el monitoreo clásico no puede ver.

*II-C3. OpenTelemetry como Estándar de Observabilidad:* OpenTelemetry (OTel) [8] es un marco open source respaldado por la CNCF que provee APIs, SDKs y un colector agnóstico al proveedor (*vendor-neutral*) para recolectar métricas, logs y trazas. Su arquitectura de *pipeline* permite recibir telemetría de múltiples fuentes, procesarla y exportarla a distintos backends de análisis como Grafana, Jaeger o Prometheus. En ciberseguridad, su aporte clave es la **trazabilidad distribuida**: un TraceID único inyectado en el punto de entrada se propaga a través de cada microservicio, permitiendo reconstruir causalmente el recorrido de cualquier solicitud post-autenticación [9], [21].

*II-C4. eBPF como Capa de Observabilidad de Kernel:* eBPF (*Extended Berkeley Packet Filter*) [10] permite ejecutar programas personalizados directamente en el kernel Linux sin recompilarlo, con una sobrecarga hasta 210 veces menor que agentes en user space. Complementa a OpenTelemetry con visibilidad a nivel de sistema operativo: captura syscalls, conexiones de red y operaciones de disco sin requerir modificaciones al código fuente [11], [12]. Su implementación completa queda fuera del alcance del presente trabajo y se propone como extensión futura.

### *II-D. Niveles de Garantía de Autenticación: NIST SP 800-63-4*

El NIST publicó en 2025 la cuarta revisión de sus Directrices de Identidad Digital, estableciendo tres niveles de garantía (AAL) [4]:

## III. Trabajos Relacionados

Esta sección examina el estado del arte en tres áreas que convergen en la propuesta: la medición del panorama de amenazas, las soluciones industriales de autenticación moderna y los avances en observabilidad orientada a seguridad.

### *III-A. Panorama de Amenazas y Estudios de Impacto*

Verizon [2] documenta en su DBIR 2025 que en promedio el 19 % de los intentos de autenticación diarios corresponde a ataques de *credential stuffing*, alcanzando picos del 44 %

[1] AiTM: *Adversary-in-the-Middle*, donde el atacante intercepta contraseña y OTP en tiempo real.

Cuadro I
NIVELES AAL SEGÚN NIST SP 800-63-4

| AAL | Requisitos | Aplicaciones Típicas |
|---|---|---|
| 1 | Un solo factor. Desaconsejado para datos sensibles. | Apps públicas de bajo riesgo. |
| 2 | MFA obligatoria. Passkeys sincronizables autorizados explícitamente. | Nueva línea base para apps corporativas. |
| 3 | Máxima garantía. Claves ligadas al hardware (TPM, Secure Enclave). Prohíbe passkeys sincronizables. | Infraestructura gubernamental y financiera crítica. |

en campañas coordinadas, y concluye que la reutilización de credenciales filtradas es el vector de entrada más frecuente en brechas de datos.

AuthX [1] estima las pérdidas globales por ATO en 9,5 billones de dólares anuales para 2025. En el contexto colombiano, ACIS [27] reporta más de 36.000 millones de intentos de ciberataque durante 2024, mientras que la CCIT [28] documenta un crecimiento del 23 % interanual en denuncias por delitos informáticos. Estos estudios establecen la urgencia del problema pero no proponen arquitecturas de solución que integren autenticación y observabilidad sobre stacks tecnológicos específicos.

### *III-B. Autenticación Moderna: Propuestas de la Industria*

MojoAuth [3] documenta tasas de éxito superiores al 98 % y reducciones de fricción del 80 % con passkeys FIDO2/WebAuthn, pero no aborda la integración con frameworks de backend específicos ni con flujos OAuth 2.0. Frontegg [13] propone una arquitectura JWT con RBAC para aplicaciones SaaS, pero asume un IdP externo. HYPR [15] revisa el NIST SP 800-63-4 y establece la distinción entre passkeys sincronizables (AAL2) y claves ligadas al hardware (AAL3). Srivathsav [14] cubre OAuth 2.0 con PKCE para clientes públicos, pero no integra WebAuthn ni observabilidad. Ninguno de estos trabajos aborda la integración conjunta de autenticación moderna y observabilidad transversal sobre Spring Boot y React.

### *III-C. Observabilidad Orientada a Seguridad*

New Relic [18], CrowdStrike [19] y Dynatrace [20] establecen de forma convergente que la ausencia de trazabilidad distribuida es el factor que permite a los atacantes moverse lateralmente sin ser detectados tras un compromiso de credenciales, y proponen el marco MELT como estándar de la industria.

Horovits [9] presenta OpenTelemetry con eBPF como la evolución hacia observabilidad de cero código. La Linux Foundation [10] documenta que eBPF ha alcanzado madurez de producción en entornos cloud-native. Doe et al. [11] demuestran una reducción de overhead del 95 % frente a agentes tradicionales en microservicios.

The New Stack [21] identifica la correlación entre identidad y telemetría post-autenticación como el gap más relevante en plataformas de observabilidad actuales. Este gap, junto con la ausencia de una arquitectura que integre login seguro y observabilidad transversal sobre Spring Boot, React y conformidad con el marco normativo colombiano, justifica la propuesta del presente trabajo.

## IV. MARCO LEGAL COLOMBIANO

### *IV-A. Ley 1273 de 2009 — Delitos Informáticos*

Esta ley modificó el Código Penal para tipificar los delitos informáticos [29]. Los artículos más relevantes son:

- **Art. 269A (Acceso abusivo):** Sanciona el acceso sin autorización. Exige autenticación ligada a un sistema de autorización robusto (RBAC) que impida la escalada de privilegios. La trazabilidad OTel provee la evidencia forense para su persecución.
- **Art. 269G (Suplantación de sitios web):** Penaliza páginas fraudulentas para capturar credenciales. Los passkeys responden directamente vinculando la autenticación al dominio real del servicio.
- **Art. 269I (Hurto por medios informáticos):** Sanciona suplantar a un usuario ante sistemas de autenticación para cometer fraude.

### *IV-B. Ley 1581 de 2012 — Protección de Datos Personales*

Esta ley regula el derecho fundamental al *Habeas Data* [30]. Sus principios rectores imponen obligaciones sobre autenticación y observabilidad:

- **Finalidad:** Los datos telemétricos recopilados por OTel deben usarse exclusivamente para los fines de seguridad informados al usuario.
- **Seguridad:** Obliga a implementar medidas técnicas para evitar adulteración, pérdida o acceso no autorizado a tokens de sesión y datos de telemetría.
- **Libertad:** El tratamiento de datos requiere consentimiento previo, expreso e informado del titular.

## V. ARQUITECTURA PROPUESTA

La propuesta integra dos dimensiones inseparables: un sistema de login seguro sin contraseñas y una capa de observabilidad transversal que atraviesa todos los componentes del sistema.

### *V-A. Visión General*

La arquitectura establece dos planos de operación complementarios:

- **Plano de autenticación:** El cliente React/Vite inicia los flujos de login, gestiona tokens en memoria y comunica con Spring Boot vía HTTPS. El servidor valida credenciales criptográficas, emite JWT y aplica RBAC sobre cada recurso protegido.
- **Plano de observabilidad:** El OTel Collector actúa como agregador central de telemetría. Cada componente — React, Spring Boot, almacenes de datos — emite trazas, métricas y logs hacia el Collector, que los procesa y exporta a Grafana mediante el protocolo OTLP. Esta capa es transversal: no

pertenece al cliente ni al servidor, sino que los atraviesa a ambos.

La infraestructura de despliegue es cloud (AWS o Azure), donde el OTel Collector y Grafana se despliegan como servicios independientes del stack de aplicación, garantizando que la telemetría persista incluso ante fallas en los componentes de negocio.

### *V-B. Modelo de Amenazas STRIDE*

El análisis de amenazas se realiza aplicando STRIDE (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*) [7] sobre los componentes principales de la arquitectura.

Cuadro II
ANÁLISIS STRIDE SOBRE LA ARQUITECTURA PROPUESTA

| Amenaza | Vector concreto | Control en la arquitectura |
|---|---|---|
| **S**poofing | Suplantación de identidad mediante credenciales robadas o phishing. | Passkeys WebAuthn: clave privada en enclave seguro, firma vinculada al dominio real. |
| **T**ampering | Modificación de JWT en tránsito o manipulación del payload. | Firma RS256 con clave RSA en servidor; TLS 1.3 cifra el canal. |
| **R**epudiation | Negación de acciones post-autenticación. | Auditoría Spring Security + trazas OTel con TraceID por sesión, exportadas a Grafana con retención configurable. |
| **I**nformation **D**isclosure | Extracción de JWT desde el cliente vía XSS. | Tokens solo en memoria React; CSP restrictiva; TLS 1.3. |
| **D**enial of **S**ervice | Flooding del endpoint de autenticación. | Rate limiting en Spring Security; alertas en Grafana ante anomalías de volumen detectadas por OTel. |
| **E**levation of **P**rivilege | Token con rol bajo accediendo a recursos de mayor privilegio. | RBAC con `@PreAuthorize` y scopes granulares (`blueprints.read`, `blueprints.write`). |

La capa OTel refuerza directamente las categorías de **Repudiation** (trazas inmutables con TraceID por sesión) y **Denial of Service** (alertas en Grafana ante patrones anómalos de volumen en endpoints de autenticación), cerrando las dos categorías que en arquitecturas sin observabilidad quedan parcialmente cubiertas.

### *V-C. Pilar 1 — Passkeys / WebAuthn*

Mitigando la amenaza de **Spoofing**, la integración de WebAuthn en Spring Boot se realiza mediante la librería `java-webauthn-server` de Yubico. Cada solicitud de registro y autenticación genera eventos OTel con el TraceID de sesión, permitiendo correlacionar en Grafana el flujo completo de autenticación con las acciones posteriores del usuario.

**Registro:**

1. El cliente solicita *PublicKeyCredentialCreationOptions* con challenge aleatorio al servidor.
2. El navegador invoca `navigator.credentials.create()`; el dispositivo genera el par de claves y retorna la clave pública.
3. El servidor verifica y persiste la clave pública; OTel registra el evento de registro con timestamp e identidad.

**Autenticación:**

1. El servidor emite *PublicKeyCredentialRequestOptions* con nuevo challenge.
2. El dispositivo firma el challenge tras verificación biométrica.
3. El servidor verifica la firma y emite un JWT; OTel emite una traza con el TraceID que acompañará todas las solicitudes de esa sesión.

### *V-D. Pilar 2 — OAuth 2.0 + PKCE*

Abordando **Information Disclosure** sobre el código de autorización, se implementa OAuth 2.0 Authorization Code con PKCE usando Spring Authorization Server. El cliente React genera el `code_verifier` en memoria, nunca en `localStorage`. El OTel SDK de React instrumenta automáticamente las solicitudes HTTP del flujo OAuth, propagando el TraceID entre el cliente y el servidor de autorización.

### *V-E. Pilar 3 — JWT con Spring Security*

Cerrando **Tampering** y **Elevation of Privilege**, el servidor emite JWT firmados con RS256. Spring Security intercepta cada solicitud, verifica la firma y construye el `SecurityContext`. El SDK de OTel para Spring Boot instrumenta automáticamente cada endpoint protegido, registrando identidad del usuario, scope validado y resultado de autorización como atributos de la traza, visibles en Grafana sin instrumentación manual.

### *V-F. Capa de Observabilidad Transversal*

La capa de observabilidad es el componente que diferencia esta arquitectura de propuestas previas: no es un añadido al final del pipeline, sino una dimensión transversal que atraviesa todos los componentes desde el diseño.

*V-F1. OTel Collector como Agregador Central:* El OTel Collector se despliega como servicio independiente en la infraestructura cloud (AWS o Azure). Recibe telemetría de tres fuentes simultáneas:

- **React/Vite:** El SDK `@opentelemetry/sdk-trace-web` instrumenta automáticamente las solicitudes HTTP del frontend, capturando latencias, errores y el TraceID de cada interacción del usuario.
- **Spring Boot:** El agente Java de OTel se adjunta al proceso sin modificar el código fuente, capturando cada endpoint REST, consulta a base de datos y evento de Spring Security.
- **Almacenes de datos:** Los logs de auditoría de DS3 se exportan al Collector mediante el protocolo OTLP.

*V-F2. Pipeline de Seguridad en Grafana:* El Collector exporta la telemetría unificada a Grafana vía OTLP, donde se configuran tres tipos de visualizaciones orientadas a seguridad:

1. **Dashboard de autenticación:** Tasa de intentos de login por IP, latencia del flujo WebAuthn y tasa de éxito/fallo por endpoint. Una caída brusca en la tasa de éxito combinada con picos de tráfico activa alertas automáticas de posible *credential stuffing*.

2. **Trazas de sesión:** Cada TraceID permite reconstruir en Grafana el recorrido completo de una sesión: desde el login hasta cada recurso accedido, con timestamps e identidad del usuario. Esto provee evidencia forense directamente admisible ante incidentes de seguridad.
3. **Alertas comportamentales:** Patrones anómalos en la frecuencia o distribución geográfica de solicitudes autenticadas disparan alertas configurables, cubriendo la categoría A09:2021 de OWASP que el monitoreo tradicional no puede detectar.

### *V-G. Seguridad en el Frontend React/Vite*

Las aplicaciones React/Vite operan en un entorno donde XSS y CSRF son las vulnerabilidades predominantes [22]:

- **Sin almacenamiento de tokens en localStorage:** Los JWT se mantienen exclusivamente en memoria de la aplicación React.
- **Cabeceras de seguridad HTTP:** TLS 1.3, HSTS y CSP restrictiva configurada en Spring Boot [23].
- **Protección CSRF:** Endpoints stateless protegidos por JWT son inherentemente resistentes a CSRF.

## VI. Evaluación y Conformidad

### *VI-A. Mitigación del OWASP Top 10*

- **A01:2021 — Broken Access Control:** Mitigado mediante RBAC con Spring Security y scopes JWT granulares por recurso.
- **A02:2021 — Cryptographic Failures:** Mitigado con RS256, TLS 1.3 y criptografía asimétrica WebAuthn en el dispositivo.
- **A07:2021 — Identification and Authentication Failures:** Erradicado con passkeys FIDO2/WebAuthn, eliminando contraseñas almacenadas en servidores.
- **A09:2021 — Security Logging and Monitoring Failures:** Cubierto completamente mediante la capa OTel + Grafana, que provee trazabilidad distribuida por sesión, dashboards de seguridad en tiempo real y alertas comportamentales configurables sobre todos los componentes de la arquitectura.

### *VI-B. Alineación con NIST SP 800-63-4*

- **AAL2:** Los passkeys sincronizables satisfacen los requisitos de MFA resistente al phishing para aplicaciones comerciales. Este es el nivel objetivo de la arquitectura propuesta.
- **AAL3:** Alcanzable reemplazando passkeys sincronizables por claves ligadas al hardware (TPM, Secure Enclave no exportable).

### *VI-C. Cumplimiento del Marco Legislativo Colombiano*

- **Ley 1273/2009 [29]:** La arquitectura FIDO2 impide algorítmicamente la suplantación del Art. 269G. Las trazas OTel exportadas a Grafana proveen cadena de custodia digital para la persecución del acceso abusivo (Art. 269A).
- **Ley 1581/2012 [30]:** La eliminación de contraseñas y la no persistencia de tokens en el cliente reducen el riesgo de violación del Habeas Data. La telemetría OTel se limita a fines de seguridad, cumpliendo el principio de finalidad.

### *VI-D. Limitaciones de la Arquitectura*

- **Cobertura parcial de DoS:** El rate limiting en Spring Security y las alertas de Grafana mitigan ataques de baja-media intensidad, pero no constituyen defensa suficiente ante ataques volumétricos coordinados. La protección completa requeriría un WAF externo.
- **Dependencia del enclave del dispositivo:** La seguridad de los passkeys depende de la integridad del Secure Enclave o TPM del dispositivo. Dispositivos comprometidos a nivel de firmware quedan fuera del modelo de amenazas.
- **Compatibilidad WebAuthn:** La API WebAuthn requiere navegadores modernos y dispositivos con capacidad biométrica o llaves FIDO. Usuarios con hardware legado requieren un mecanismo alternativo durante la transición.
- **Privacidad de la telemetría:** La capa OTel recopila datos comportamentales de sesión que deben tratarse con estricto cumplimiento del principio de finalidad de la Ley 1581/2012, limitando su uso exclusivamente a fines de seguridad.
- **Alcance de la validación:** La conformidad con OWASP, NIST y el marco legal colombiano se evalúa mediante análisis arquitectónico. La arquitectura no ha sido sometida a pruebas de penetración formales ni auditorías de terceros.

## VII. Conclusiones

La seguridad de aplicaciones en 2025 requiere abordar simultáneamente dos problemáticas que se retroalimentan: la debilidad del mecanismo de login y la ceguera operacional que permite a los atacantes operar sin ser detectados. Una arquitectura que resuelve solo una de las dos es incompleta.

La propuesta presentada demuestra que ambas problemáticas son abordables de forma integrada sobre un stack de desarrollo moderno. Los passkeys FIDO2/WebAuthn erradican algorítmicamente el *credential stuffing* y el phishing a nivel de protocolo, con tasas de éxito superiores al 98 %. OAuth 2.0 con PKCE garantiza que los flujos de autorización en clientes React sean seguros ante interceptación. JWT con Spring Security provee control de acceso stateless y matemáticamente verificable.

La capa de observabilidad transversal basada en OpenTelemetry y Grafana transforma la postura de seguridad de reactiva a proactiva: los TraceIDs por sesión correlacionan la identidad del usuario con cada acción post-autenticación, los dashboards de Grafana detectan patrones de *credential stuffing* en tiempo real, y las trazas inmutables proveen cadena de custodia forense ante incidentes. Esta capa cierra completamente la categoría A09:2021 de OWASP, que el monitoreo tradicional es estructuralmente incapaz de cubrir.

La arquitectura alcanza AAL2 del NIST SP 800-63-4, aborda A01, A02, A07 y A09 del OWASP Top 10, y satisface los principios de seguridad y finalidad de la Ley 1581 de 2012, demostrando que login seguro, observabilidad transversal y cumplimiento normativo no son objetivos contrapuestos sino componentes sinérgicos de una misma estrategia de seguridad por diseño.

## Referencias


[1] AuthX, "Account Takeover Statistics 2025: Key Trends & Insights," *AuthX Blog*, 2025. [Online]. Available: https://www.authx.com/blog/account-takeover-fraud-statistics-insights/

[2] Verizon, "Additional 2025 DBIR Research on Credential Stuffing," *Verizon Business Resources*, 2025. [Online]. Available: https://www.verizon.com/business/resources/articles/credential-stuffing-attacks-2025-dbir-research/

[3] MojoAuth, "Passkeys Handbook 2025: Secure, Passwordless Authentication," *MojoAuth White Papers*, 2025. [Online]. Available: https://mojoauth.com/white-papers/passkeys-passwordless-authentication-handbook/

[4] NIST, "Digital Identity Guidelines: SP 800-63-4," *NIST Special Publication 800-63-4*, 2025. [Online]. Available: https://pages.nist.gov/800-63-4/sp800-63.html

[5] NIST NCCoE, "Implementing a Zero Trust Architecture," 2025. [Online]. Available: https://www.nccoe.nist.gov/projects/implementing-zero-trust-architecture

[6] OWASP Foundation, "OWASP Top Ten 2021," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[7] OWASP Foundation, "OWASP Secure by Design Framework," 2024. [Online]. Available: https://owasp.org/www-project-secure-by-design-framework/

[8] OpenTelemetry, "OpenTelemetry: An Observability Framework for Cloud-Native Software," *CNCF*, 2025. [Online]. Available: https://opentelemetry.io/

[9] D. Horovits, "Unveiling OpenTelemetry eBPF Instrumentation," *Medium*, 2025. [Online]. Available: https://horovits.medium.com/unveiling-opentelemetry-ebpf-instrumentation-377cb0432bf1

[10] Linux Foundation, "The State of eBPF 2025," 2025. [Online]. Available: https://www.linuxfoundation.org/hubfs/eBPF/The_State_of_eBPF25_111925.pdf

[11] J. Doe *et al.*, "eBPF-Enhanced Complete Observability Solution for Cloud-Native Microservices," *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/379891642

[12] Groundcover, "OpenTelemetry and eBPF: Everything You Need to Know," *Groundcover Blog*, 2025. [Online]. Available: https://www.groundcover.com/ebpf/opentelemetry-ebpf

[13] Frontegg, "What Is User Authentication? A 2025 Guide for Modern Apps," *Frontegg Blog*, 2025. [Online]. Available: https://frontegg.com/blog/authentication

[14] J. R. S. Adari, "Building Secure Authentication Layers: A Complete Guide for 2025," *srivathsav.me*, 2025. [Online]. Available: https://srivathsav.me/blog/building-secure-authentication-layers-2025

[15] HYPR, "NIST SP 800-63-3 & 63-4: Digital Identity Guidelines Review," *HYPR Blog*, 2025. [Online]. Available: https://www.hypr.com/blog/nist-sp-800-63-3-digital-identity-guidelines-review

[16] Authsignal, "Passwordless Authentication in 2025: The Year Passkeys Went Mainstream," *Authsignal Blog*, 2025. [Online]. Available: https://www.authsignal.com/blog/articles/passwordless-authentication-in-2025-the-year-passkeys-went-mainstream

[17] Google Cloud, "Modern Password Security for System Designers," *Google Cloud Documentation*, 2024. [Online]. Available: https://docs.cloud.google.com/solutions/modern-password-security-for-system-designers.pdf

[18] New Relic, "Observability vs. Monitoring: What's the Difference?" *New Relic Blog*, 2025. [Online]. Available: https://newrelic.com/blog/observability/observability-vs-monitoring

[19] CrowdStrike, "Observability vs. Monitoring: What's the Difference?" *CrowdStrike Cybersecurity 101*, 2025. [Online]. Available: https://www.crowdstrike.com/en-us/cybersecurity-101/observability/observability-vs-monitoring/

[20] Dynatrace, "Observability vs. Monitoring: What's the Difference?" *Dynatrace Blog*, 2025. [Online]. Available: https://www.dynatrace.com/news/blog/observability-vs-monitoring/

[21] The New Stack, "Observability in 2025: OpenTelemetry and AI to Fill In Gaps," *The New Stack*, 2025. [Online]. Available: https://thenewstack.io/observability-in-2025-opentelemetry-and-ai-to-fill-in-gaps/

[22] StackHawk, "10 Web Application Security Threats for 2025," *StackHawk Blog*, 2025. [Online]. Available: https://www.stackhawk.com/blog/10-web-application-security-threats-and-how-to-mitigate-them/

[23] Oligo Security, "Web App Security in 2025: 5 Technologies You Can't Do Without," *Oligo Academy*, 2025. [Online]. Available: https://www.oligo.security/academy/web-app-security-in-2025-5-technologies-you-cant-do-without

[24] Northern Technologies Group, "Zero Trust Architecture in 2025," *NTG Blog*, 2025. [Online]. Available: https://ntgit.com/zero-trust-architecture-in-2025-shifting-from-perimeter-security-to-never-trust-always-

[25] CLEAR, "Fraud Analytics in 2025: Why Traditional Systems Fail," *CLEAR Identity Blog*, 2025. [Online]. Available: https://identity.clearme.com/post/fraud-analytics-2025-why-traditional-systems-fail

[26] DigitalOcean, "Understanding AI Fraud Detection and Prevention in 2026," *DigitalOcean Resources*, 2025. [Online]. Available: https://www.digitalocean.com/resources/articles/ai-fraud-detection

[27] ACIS, "Colombia enfrenta récord de 36.000 millones de ciberataques en 2024," *ACIS*, 2024. [Online]. Available: https://www.acis.org.co/blog/noticias-2/colombia-enfrenta-record-de-36000-millones-de-ciberataques-en-2024-4964

[28] CCIT, "Balance de Ciberseguridad 2024," *CCIT*, 2024. [Online]. Available: https://www.ccit.org.co/noticias/balance-de-ciberseguridad-2024-desafios-y-prevencion-para-un-entorno-digital-seguro

[29] Congreso de la República de Colombia, "Ley 1273 de 2009: Protección de la información y de los datos," 2009. [Online]. Available: https://www.armada.mil.co/sites/default/files/normograma_arc/telematica/Ley%201273%20de%202009%20Modifica%20CP%20para%20protecci%C3%B3n%20de%20datos%20e%20informaci%C3%B3n.pdf

[30] Congreso de la República de Colombia, "Ley 1581 de 2012: Protección de datos personales," 2012. [Online]. Available: https://www.alcaldiabogota.gov.co/sisjur/normas/Norma1.jsp?i=49981

[31] International Organization for Standardization, "ISO/IEC 27001:2022 – Information Security Management Systems," *ISO*, 2022.